Personal AI Hackathon Foundation Research

Architecture and build decisions for Iron Web

23 October 2026
Nebius x NVIDIA Global AI Hackathon
Personal AI track

Recommended foundation
Hermes with user-controlled durable memory, Nemotron Lightning and Ultra through Nebius Token Factory, and a pinned sandbox integration after validation.

Includes architecture choices, privacy boundaries, deployment tradeoffs, cost scenarios, acceptance tests, a submission plan and a source coverage ledger.

Research and design proposal. Application tests were not executed in this task. No slide deck was supplied; documentation coverage is recorded explicitly.

Contents

1 Recommended foundation 3
2 Research scope and confidence 4
3 Hackathon requirements and participation 4
4 Product definition and differentiation 5
5 What each technology contributes 6
6 Architecture options and tradeoffs 7
7 Proposed system and data flow 9
8 Memory design and existing system integration 10
9 Skills and controlled learning 11
10 Nemotron model strategy 12
11 Token Factory API integration 13
12 Security and privacy design 14
13 Tools and MCP integration 16
14 Always on operation and scheduling 16
15 State durability and recovery 17
16 Nebius deployment choices 18
17 Costs and budget controls 19
18 Test plan and release criteria 20
19 Main risks and fallback decisions 22
20 Build plan through submission 23
21 Decisions still requiring validation 25
22 Build prerequisites and starter runbook 25
23 Source ledger and review coverage 28

1 Recommended foundation

Build a personal research and follow-through assistant around a memory service the user can inspect, correct, export, and delete. Use Hermes as the agent runtime, Nemotron 3.5 Lightning for routine work, and Nemotron 3 Ultra for bounded difficult tasks, both through Nebius Token Factory. Make the first complete workflow a research brief that remembers decisions and follows up on an explicitly chosen task.

Treat NemoClaw as the preferred managed sandbox integration if its pinned stack passes a short integration trial. Keep a simpler Hermes deployment as the delivery fallback. The differentiator should be useful, accountable memory and reliable follow-through. Installing several frameworks does not establish a useful product.

This is a proposed foundation, not a certification of the existing memory_system. Parent-conversation references to Iron Web establish context only. This research did not run its tests, inspect its current implementation, use credentials, provision cloud resources, or change the main project's source. All acceptance tests below are proposed work unless explicitly described as document checks.

The decisions to make first

Decision	Recommendation	Reason and condition
User and problem	One researcher or independent builder tracking a project	A specific workflow gives memory and proactive assistance an observable purpose
Agent	Hermes	Existing memory, skills, sessions, tools, and scheduler reduce orchestration work
Inference	Both requested Nemotron models on Token Factory	Directly connects the required model and platform to useful runtime work
Durable memory	One authoritative service controlled by the user	Prevents conflicting copies across agent notes, graph indexes, and transcripts
Sandbox	Pinned NemoClaw with Hermes after a compatibility trial	Managed policy and credential handling are valuable; preview integration creates delivery risk
Persistent host	A supported Linux host kept available	Laptop sleep and manual reboot recovery can interrupt background tasks
Cloud compute	Optional finite evaluations or indexing jobs	Token Factory already supplies the required Nebius runtime dependency
Public demo	Separate synthetic dataset with a resettable workspace	Demonstrates privacy controls without publishing personal records

Conditions for committing to the stack

Before building the full UI, verify that the account can call both exact model IDs, that a tool call completes through the intended sandbox, that the memory service is reachable under enforced policy, and that a restarted process can retrieve an earlier fact. Time-box this trial to half a day. If it fails, use direct Hermes with narrowly exposed tools and preserve the same memory API. This keeps the application portable.

2 Research scope and confidence

The supplied URLs are documentation portals and a hackathon website. No slide deck was supplied. The portals include hundreds of pages, generated API references, release histories, and unrelated cloud products. This report reviews the foundation-relevant material and records its coverage in Section 23. It does not claim that every page, linked video, slide, API schema, or version of those sites was read.

Primary evidence comes from NVIDIA, Nous Research, Nebius, and the official Devpost event pages. Documentation indexes were used to discover relevant topics. A page in an index was not counted as a page reviewed. Long pages were reviewed by substantive sections; navigation menus and index listings are not evidence of implementation behavior.

Verified means an official page supports the statement as accessed on the research date. **Recommendation** means an engineering choice proposed here. **Validate** means account access, integration behavior, hardware compatibility, or contractual terms still require a check. Dynamic catalogs and pricing can change after this report.

Several Markdown documentation routes failed in the research tool, so readable HTML pages were used. The Token Factory model catalog is a JavaScript application that yielded no readable body; official model marketing pages, catalog link targets, the Nebius cookbook, and the documented discovery API provide supporting evidence. No authenticated catalog call was made.

The broad Nebius cloud portal was scoped to serverless compute, storage lifecycle, pricing and quotas, and deployment prerequisites. Kubernetes operations, Slurm training clusters, physical AI, and fine-tuning recipes were not exhaustively audited because they are outside the recommended minimum product.

3 Hackathon requirements and participation

Submission essentials

The project needs an NVIDIA open model and qualifying Nebius runtime use. A working URL or test build, public licensed repository with setup instructions, description, track selection, public YouTube demonstration under three minutes, and technology feedback are submission essentials. Existing projects need an explanation of substantial changes made during the submission period. The named agent tools are examples rather than a requirement to use all of them. [S01]

The official rules define qualifying use as a runtime Token Factory call or the listed AI Cloud deployment paths. Keep testing access free through the end of judging. Submission language must be English or

accompanied by translation. Check eligibility, ownership, third-party licenses, conflict exclusions, publicity terms, and prize paperwork personally before entry. Rules prevail over conflicting promotional text. [S02]

Dates and participation facts

Milestone	Published Pacific time	Practical implication
Submission opens	26 August 2026 at 09:00 PT	Record which existing work predates this period
Submission deadline	30 October 2026 at 10:00 PDT	17:00 UTC and 22:30 India time on 30 October
Judging	1 December 2026 at 09:00 PT to 15 December at 12:00 PT	Budget for continued demo availability
Winners expected	Around 11 January 2027 at 12:00 PT	Announcement timing can change

Dates above come from the official rules. [S02]

The FAQ allows solo entry and teams without a maximum size. It says the rest of the application can live elsewhere when an NVIDIA model is called through Token Factory. An NVIDIA API key alone does not meet the Nebius requirement. Country onboarding problems need resolution with Nebius; the FAQ explicitly rejects using a different billing region as a workaround. [S03]

Credits and awards

The resource page advertises $25 of Token Factory credit through its direct credits form using NEBIUS-DEVPOST-GLOBAL26, plus another $25 through the Builders Program. Approval and delivery are not guaranteed. The FAQ clarifies that these are Token Factory credits, with no separate published AI Cloud grant. Verify your actual balance and expiry before treating credits as budget. [S04] [S03]

The Personal AI track award is a Jetson Orin Nano. Overall awards are $20,000, $10,000, and $6,000. [S01] City awards require in-person attendance according to the FAQ; a conflicting sentence on the resources page should not be used to claim eligibility. [S03]

The four judging categories are equally weighted: implementation, design, impact, and quality of idea. [S02] Allocate effort accordingly: a comprehensible memory correction flow, a concrete user problem, and a reliable working demonstration deserve substantial time alongside backend engineering.

4 Product definition and differentiation

Proposed product

Iron Web helps a person continue work across days without repeatedly explaining their projects. It creates short research briefs from chosen sources, retains user-approved facts and decisions, and follows up on a selected commitment. Every durable assertion has a visible origin and can be corrected.

Example: the user says, “For this project, prioritize privacy over latency. Review these three papers and prepare a brief tomorrow morning.” The assistant saves the preference with consent, reads the chosen material, proposes a reusable review procedure, and schedules the brief. On the next day it produces a source-linked draft. If the user changes the priority, subsequent recommendations reflect the newer decision.

This is a proposed use case, not evidence of customer demand. Validate it with three short user interviews and one real task from each person. Ask what they currently forget, which sources they trust, what they would let an assistant do unattended, and what would make them stop using it.

Minimum complete experience

1. Connect a selected folder or upload a small document set with a clear scope preview.
2. Ask a question and receive an answer with source references and uncertainty.
3. Save a preference or decision, inspect its record, then retrieve it in a fresh session.
4. Correct a stale fact and show the replacement in the next answer.
5. Schedule one recurring or one-shot brief with a visible next-run time and pause control.
6. Create a draft artifact; require approval before any external send or irreversible action.
7. Show which data was sent to which model and why.
8. Export and delete the selected memory with a testable receipt.

Scope to defer

Defer broad autonomous browsing, many messaging channels, multi-user sharing, fine-tuning, voice, shopping, payments, and generalized computer control. Each adds credentials, failure modes, or evaluation work. The first release should complete one workflow repeatedly, survive restart, and provide useful evidence of its behavior.

Hermes already offers memory and skill interfaces, including a learning timeline. A generic memory graph alone is therefore a weak novelty claim. [S11] Differentiate through temporal correctness, complete deletion across stores, controlled inference disclosure, and measured usefulness in the chosen workflow.

5 What each technology contributes

Component	Job	Main upside	Main downside
Hermes Agent	Agent loop, tools, skills, sessions, messaging and scheduling	Saves substantial product plumbing	Built-in behaviors and background model calls need configuration and evaluation

Component	Job	Main upside	Main downside
OpenClaw	Alternative agent runtime in NemoClaw	The default reference integration	Switching runtimes introduces separate memory and configuration behavior
NemoClaw	Onboarding, blueprint, policy integration and lifecycle management	Provides a managed reference stack	Alpha status, platform caveats and version coupling
OpenShell	Execution boundary and policy enforcement	Separates workload access from provider secrets	Allowed destinations and trusted services remain important escape paths
Nemotron models	Reasoning, extraction, planning and tool selection	Open model family with two useful capability tiers	Output can be wrong, malformed, slow or expensive across many turns
Token Factory	Hosted inference API	No model-serving cluster to operate	Data leaves the local machine; shared endpoint region is not fixed
Nebius AI Cloud Serverless	Run containers as endpoints, jobs or interactive environments	Optional custom compute and evaluations	Separate billing, quotas and storage lifecycle
Iron Web memory service	Proposed user-controlled durable knowledge layer	Can provide provenance, corrections and explicit deletion	Its correctness and availability become application responsibilities

Hermes, OpenShell and NemoClaw occupy different layers. NemoClaw operates a reference integration that runs Hermes in OpenShell; it does not replace Hermes's agent logic. Using OpenShell directly means supplying the image, policies, provider setup and integration yourself. [S05]

Choose one agent runtime for the minimum product. Do not run Hermes inside OpenClaw or maintain two independent schedulers for the same workflow. Keep a narrow application layer above the chosen runtime so a later replacement does not require rewriting the memory service.

6 Architecture options and tradeoffs

Option A Managed NemoClaw and Hermes

Use a supported Linux host, a reviewed NemoClaw version, its compatible OpenShell release, Hermes, the memory API, and Token Factory. This offers a strong demonstration of credential isolation and declarative policy with less custom sandbox assembly.

The cost is operational complexity. NemoClaw is early-preview alpha, provides no production SLA, and has documented limitations for Hermes compatibility and host support. Linux Docker on Ubuntu 24.04 is the primary tested path; Windows WSL2 is tested with limitations. Native Hermes Windows support does

not imply native NemoClaw Windows support. [S06]

Choose this if a clean integration trial passes and you can show actual policy denials, successful allowed tool calls, state restoration, and a documented reboot runbook. Treat continuous availability as something to demonstrate, not an automatic property of the framework.

Option B Hermes with a simpler deployment

Run Hermes on a supported host using its documented installation method. Expose a narrow memory API and task-specific tools. Use its own service and scheduler support, with operating-system and network controls appropriate to that deployment.

This reduces managed-stack dependencies and can be the fastest route to a complete product. It also transfers more isolation, credential handling and service ownership to you. Hermes supports native Windows, Linux/WSL2, macOS Apple Silicon and Docker as Tier 1 paths; PyPI and Homebrew installations are explicitly unsupported distribution methods. [S07]

Choose this if the managed stack blocks delivery. Describe the actual protections accurately. Ordinary Docker isolation and command approval do not establish parity with OpenShell's enforcement boundary.

Option C OpenShell directly with a custom agent image

Use OpenShell's current provider profiles and sandbox attachments, package Hermes in a custom image, and implement application-specific policy and service connections. This offers the most control over the runtime boundary and avoids assuming NemoClaw supports every topology.

It requires image maintenance, provider profiles, migration work, and more integration tests. It is appropriate if sandbox integration is central to the project's contribution and the team already understands OpenShell. It is a poor default for a solo builder with seven days remaining.

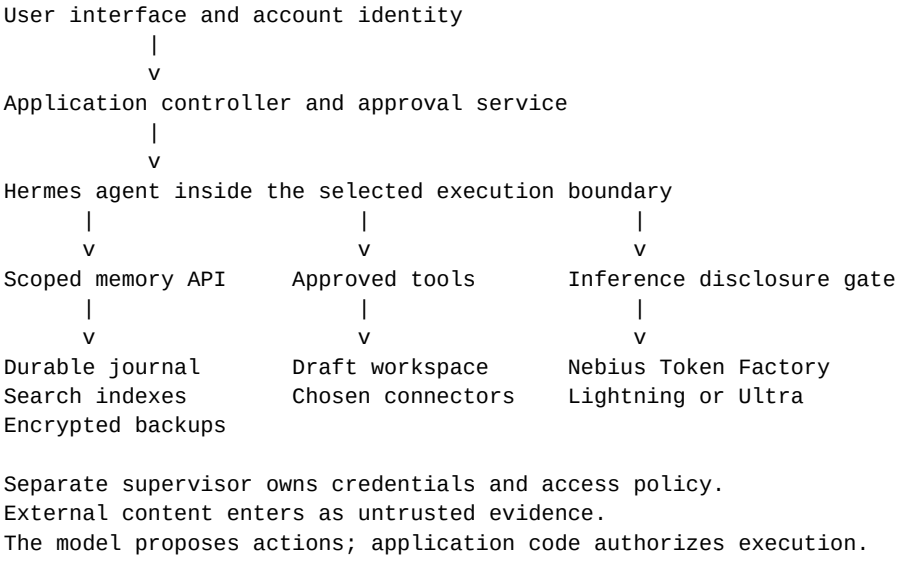
Option D Fully local models and memory

Local inference can keep prompts on user-owned hardware, but hardware, serving compatibility, model storage and power become constraints. A local-only application still needs qualifying Nebius runtime use for this hackathon. Use the local route for sensitive requests only if the system's meaningful Nebius integration is separately present and clearly demonstrated.

Recommendation: prototype B first, attempt A within a fixed time budget, and preserve B as the fallback. Avoid C and multi-node model hosting for this submission unless they are already working and evaluated.

7 Proposed system and data flow

Components and boundaries



The diagram is a proposed logical architecture. An “inference disclosure gate” is application logic you must implement and test; it is not a claim that these frameworks automatically redact arbitrary personal data.

Request sequence

Authenticate the user, resolve the workspace and permitted tools, retrieve a bounded set of authorized memories, and assemble only the context needed for the task. Apply the selected disclosure policy before inference. Ask the chosen model for a response or structured tool proposal. Validate tool arguments and authorization in code, execute if permitted, persist the result, and show the outcome with source references.

For writes, report success only after a durable acknowledgement. For external actions, use an approval record tied to the exact payload. For retries, use an operation identifier so a repeated request cannot create duplicate tasks, memory entries, or outgoing messages.

Recommended deployment split

Keep the agent gateway, scheduler, approval service and durable memory available on a persistent host. Use Token Factory for inference. Use finite Serverless Jobs only for optional batch evaluations or rebuildable indexing work. Place public demonstration data in a separate environment from personal memory.

This split costs more operational effort than a single process but prevents a stopped disposable workload from becoming a data-loss event. Start with one host and one memory backend if those satisfy the required tests. Add services only for a measured limitation.

8 Memory design and existing system integration

What Hermes already provides

Hermes's built-in memory consists of MEMORY.md and USER.md, with default limits of 2,200 and 1,375 characters. A session starts with a frozen snapshot; later writes persist but do not refresh that snapshot until a new session. Session search uses a separate SQLite history store. Memory and skill writes can be staged for approval. [S11]

Hermes external memory integrations are additive and can synchronize transcripts, recall context and mirror writes. This expands the data boundary. Secret scrubbing does not recognize every arbitrary password or opaque secret. [S12] Do not enable an external provider just because it appears in a catalog.

Choose the authority explicitly

If Iron Web is the project's contribution, make its memory service authoritative for user facts and decisions. Keep Hermes profile memory limited to non-sensitive operating preferences, or disable its built-in stores using the documented settings after validating the integration. Avoid simultaneous unsupervised writes to both systems.

For the first version, use explicit memory tools rather than a deeply coupled automatic provider plugin. A small API is easier to test, audit and replace. Add automatic prefetch or a native provider integration only when explicit tools demonstrably miss necessary recall.

Proposed durable record

Each fact should have an immutable identifier, subject and workspace, source identifier and source location, observed time, valid-from time, optional valid-until time, sensitivity, consent state, version, and deletion state. Keep model-generated confidence separate from source authority. A confident extraction from an untrusted page is still untrusted.

Distinguish a fact, a user preference, a hypothesis, a plan, an external document claim and an execution result. A user's correction should supersede a prior preference without deleting its audit relationship. A reminder is a task with a schedule and execution state, not merely a sentence stored in memory.

Retrieval and correction

Apply user, workspace, consent and deletion filters before retrieval, then combine lexical and semantic retrieval if needed. Return a small evidence bundle with record IDs, timestamps and source references. Resolve contradictory records using explicit authority and time rules. Ask when two equally authoritative current facts conflict.

Revalidate the bundle before composing the response if a deletion or correction may have occurred during inference. Cached responses should carry a memory revision so a correction invalidates stale answers. A hidden old copy in the current conversation can still influence output; the UI should support starting a fresh session after a sensitive correction or deletion.

If the current memory system uses several stores

If the implementation has a journal, vector database and graph database, keep the journal authoritative and treat the others as rebuildable projections. Record indexing checkpoints and idempotent event IDs. Test recovery when one index is behind or unavailable. A graph relationship must not silently override a newer journal fact.

If the existing code uses FalkorDB, Qdrant, SQLite or other named engines, their specific operational and license requirements still need a repository-level audit. This report did not inspect those deployed versions. Do not assume three databases are necessary for a small personal assistant. A simpler transactional store with full-text search may be sufficient for the first dataset.

Deletion must cover every copy

Define deletion separately for source documents, extracted facts, embeddings, graph edges, summaries, transcripts, generated artifacts and backups. Use tombstones so a delayed indexer cannot reintroduce removed material. Document backup expiry and ensure restores reapply tombstones before serving queries.

“Delete memory” should not promise immediate erasure from immutable backups unless the implementation can prove that property. Show the user which copies were removed, which are queued for expiry, and whether a new session is needed to clear live context.

9 Skills and controlled learning

Hermes skills are on-demand instructions stored under its skills directory. The agent can create or alter them, and hub installations go through security scanning. A scan is useful evidence, but it does not prove that a workflow is safe for your data or correct for your tools. [S13]

Create three small reviewed skills: review a paper, produce a source-linked project brief, and check a chosen commitment. Each skill should declare its inputs, allowed data sources, output format, required tools, stop conditions, and actions requiring user approval.

Example review procedure: extract the research question, method, evidence, limitations and relevance; identify unsupported claims; cite source locations; record a tentative conclusion; propose a durable memory only when it is useful beyond the current task. Keep the document's instructions separate from the user's instructions.

Enable skill write approval during the submission build. Let the assistant propose improvements after a repeated correction, then review a visible diff before activation. Version the accepted procedure and retain a rollback path. Successful execution should not automatically authorize a wider tool scope.

Treat executable scripts in skills as software dependencies. Pin their source revision, inspect changes, restrict write locations, and test malicious inputs. Avoid runtime installation from arbitrary skill repositories during the public demo.

10 Nemotron model strategy

Exact provider identifiers

Role	Token Factory model ID	Published input price per million tokens	Published output price per million tokens
Routine work	nvidia/Nemotron-3_5-Lightning	$0.06	$0.24
Difficult bounded work	nvidia/Nemotron-3-Ultra-550b-a55b	$1.00	$3.00

These catalog link targets and prices are documented by Nebius. Its page lists Lightning as BF16 serving and Ultra as FP4 serving, with 1,024K context for both. This is provider-specific information, not proof of an account's current permissions or achievable context in every harness. [S14]

NVIDIA's corresponding model cards describe Lightning as 30B total parameters with 3B active and Ultra as 550B total with 55B active. Both use OpenMDW 1.1 rather than the application's prospective MIT or Apache license. Keep model licensing separate from application licensing. [S15] [S16]

Routing recommendation

Start with deterministic routing. Use Lightning for query classification, bounded extraction, drafting and routine retrieval. Use Ultra for reconciling evidence, difficult planning and final analysis where a small evaluation shows a material quality gain. Give each task a maximum number of tool turns and a token budget.

Do not let model-reported confidence alone trigger escalation. Prefer an invalid schema, insufficient evidence, contradictory sources, a failed verifier, or an explicit user request. A larger model is not permission to send more sensitive data. Apply the same disclosure rules before both routes.

Keep Ultra work in a short separate context where possible. Mid-session model switching can invalidate prompt caching and reread a large conversation. Configure auxiliary models deliberately; compression, title generation, reviews and routing can otherwise create extra inference and disclosure. [S17]

Compatibility checks

List models using the documented API and copy the exact case-sensitive identifiers. The verbose response can expose context, pricing and limits. [S18] Test text, actual structured tool calls, tool results, streaming, cancellation and structured JSON separately. API compatibility is not a guarantee that every feature combination works.

Both models need task-level evaluation under the actual provider precision and harness. A public benchmark is useful context, not your success rate. Nebius's October cookbook demonstrates cost measurement across Lightning, Super and Ultra and records a Deep Agents middleware workaround specific to Ultra; that workaround should not be copied into Hermes without reproducing a relevant

problem. [S19]

Hosting models yourself

Ultra BF16's official card lists multi-GPU requirements including 8 H200 or 16 H100 GPUs. [S16] Active parameter count does not equal the memory needed to store all weights. Quantized deployments have different requirements and still need runtime and workload sizing. Hosted inference is the practical starting point here. Local Lightning may become useful later after checking the exact hardware recipe, available memory, context and concurrency. [S15]

11 Token Factory API integration

The documented base URL is https://api.tokenfactory.nebius.com/v1/ and examples use NEBIUS_API_KEY. [S20] This is a separate provider credential from an NVIDIA inference key. Store it in the host's approved secret mechanism; do not paste it into prompts, public repository files or client-side JavaScript.

For NemoClaw, choose the custom OpenAI-compatible endpoint path and use the Token Factory base URL and model ID. The adapter is tested with limitations; actual endpoint behavior requires validation. Onboarding may perform inference validation calls, so include them in the disclosure and cost model. Reasoning-only validation skips tool and streaming checks and is insufficient evidence for an agent. [S21]

Proposed integration sequence

1. Verify the account and billing, discover model IDs, and save a redacted capability snapshot.

2. Call each model directly with a harmless fixed prompt.

3. Define a minimal lookup tool and require an actual tool call with valid arguments.

4. Execute a deterministic local function, send its result back, and validate the answer.

5. Repeat through the selected sandbox; check that the effective route reaches Token Factory.

6. Test a source-linked memory retrieval and a draft artifact through the complete application.

7. Inject failures before enabling schedules or external actions.

The API returns function invocation proposals; your application executes them. [S22] Parse and validate the function name, arguments, subject permissions, expected object version, and permitted operation before execution. Never dispatch arbitrary model text as a shell command.

Use JSON Schema where supported and validate outputs again locally. A syntactically valid object can still contain a wrong date, invented source or unauthorized identifier. Some structured-output documentation examples reference older models or contain simplified snippets; export and test the exact request rather than copying examples blindly. [S23]

Failures and retry policy

Treat 401/403 as an authentication or authorization problem, 404 as a model or route mismatch, malformed requests as a caller bug, 429 as throttling, and temporary 5xx or connection errors as candidates for bounded retry. Respect server retry guidance and add jitter. Cap total elapsed time. Do not retry an external write unless its idempotency behavior is known.

Token Factory limits are dynamic and actual defaults are account-visible. Sudden bursts can throttle even if a marketing page quotes high throughput. [S24] Begin with low concurrency, queue background work, and show a clear “waiting for provider” state. Preserve pending tasks through an outage.

12 Security and privacy design

The privacy claim to make

“Your durable memory stays in the storage you choose. Hosted model calls receive selected context under the disclosure policy shown in the app.” This is a defensible proposed claim if implemented and tested. “Everything stays on your machine” would be false when hosted inference or cloud tools receive content.

Nebius advertises zero-retention inference and a mode where prompts and outputs are not stored or reused for training. [S25] The current public-endpoint documentation explicitly provides no fixed processing region. A region in an older blog or a region-looking base URL is not a residency guarantee. Dedicated endpoints are the documented path when location guarantees are required. [S26]

Validate the applicable account terms, endpoint mode, observability, data-lab imports, file uploads and any support logging before using sensitive real data. Zero-retention inference does not remove your application's transcripts, backups or connector logs. A certification advertised by an infrastructure provider does not certify this application.

Proposed disclosure policy

Data class	Default behavior	User-visible control
Public sources	May go to selected hosted model	Show provider and source list
Personal project notes	Send only relevant excerpts after consent	Preview disclosure and remember a narrow choice
Sensitive documents	Local processing or explicit per-task exception	Clear explanation of destination and retention uncertainty
Credentials and recovery codes	Keep outside model context	Secret reference and scoped connector action
Untrusted external text	Treat as evidence	Never allow it to authorize tools or change policy

What the sandbox adds

OpenShell separates a trusted supervisor from the agent workload, mediates network access, and binds provider credentials to permitted endpoints. The current architecture describes kernel controls and policy checking. [S27] These controls reduce the consequences of malicious instructions but do not make an allowed API call semantically safe.

An allowlisted inference endpoint can still receive private text. An allowed email API can still send the wrong message. A trusted MCP server can return malicious content or even reflect a credential it receives. Keep application authorization, disclosure checks and exact-payload approval outside the model's discretion.

OpenShell's current security guide distinguishes audit from enforce: L7 audit mode can log violations while forwarding traffic. TLS skip also removes inspection and credential rewriting for that endpoint. [S28] Inspect effective settings and test negative cases before claiming write blocking. Restrict broad interpreter grants, wildcard destinations, package registries and general web access.

Proposed threats and controls

Threat	Failure example	Required control and evidence
Prompt injection	A paper asks the agent to export notes	Source text cannot grant authority; test an actual blocked attempt
Memory poisoning	A retrieved page rewrites a user preference	User assertions outrank external claims; staged durable writes
Secret exposure	Tool error prints a token	Supervisor-held secrets, response scrubbing and canary tests
Cross-user leakage	Guessing another workspace ID retrieves facts	Enforce identity at the memory service, before search
Allowed-host exfiltration	Data hidden inside an authorized API request	Narrow operations, bounded request bodies and application disclosure checks
Approval replay	Old approval sends changed content	Bind approval to payload digest, recipient, version and expiry
Supply-chain change	A skill update broadens behavior	Pin dependencies and review skill diffs
Host compromise	Administrator reads local memory	Encryption and host controls; sandbox cannot defend against its trusted administrator

13 Tools and MCP integration

Hermes supports MCP server discovery and tool selection. [S29] NemoClaw's managed MCP path is narrower: authenticated HTTPS Streamable HTTP endpoints, managed provider credentials, and explicit protocol policy. It does not launch an arbitrary host-side stdio bridge for you. Its current managed MCP documentation pins OpenShell 0.0.116. [S30]

Proposed memory tool contract

Expose operations such as memory_search, memory_get, memory_propose, memory_correct, memory_export and memory_delete through an authenticated service. Those names are proposed application tools, not existing built-in commands. Keep ordinary agent access to search and proposals; use a separate approval path for destructive changes and broad exports.

Every request should carry a validated identity context, workspace scope, request ID and a bounded result size. The server must derive authority from authenticated context rather than accepting a model-supplied owner_id as proof. Return typed errors for denied, missing, stale and temporarily unavailable records.

If the memory API is local to the host, container localhost is not the host's localhost. Use the documented bridge or a narrowly permitted private service endpoint. With managed MCP, verify trusted-private endpoint requirements, TLS, DNS behavior and tool filtering on the pinned version. A host-side successful request does not prove sandbox reachability.

Connector scope

Begin with selected local documents and one calendar or task source in read-only mode. Add draft creation in a separate workspace. External sending should show the exact recipient, content, attachments and consequence. Recheck scope at execution time, because a proposed action may wait while credentials or permissions change.

Prefer dedicated API tools over broad shell access for routine tasks. Restrict returned document size and remove unneeded fields. MCP transport encryption and tool descriptions do not replace authorization at the underlying service.

14 Always on operation and scheduling

Hermes runs scheduled tasks through its gateway daemon, ticking roughly every 60 seconds and creating isolated agent sessions. Its status command reports scheduler health and overdue work. Restart-safe workers depend on the deployment's service/session support; containers may degrade to workers that die with the gateway. [S31]

NemoClaw's headless guide explicitly requires manual recovery after host reboot and does not guarantee automatic restart of its gateway, sandbox and forwards. [S32] Do not claim unattended reboot recovery unless it has been separately engineered and tested within a supported deployment approach.

Define the availability promise

For the hackathon, target an assistant continuously reachable while its documented host is operating, with persistent jobs, visible last success, and recovery instructions. If reboot recovery remains manual, state that limitation in the product and submission. A laptop that sleeps cannot provide reliable unattended service.

Use one scheduler as the owner of a routine. Store timezone, next scheduled instant, last successful instant, task version and execution status. Choose a missed-run policy explicitly: run once after recovery, skip stale work, or ask the user. Do not replay every missed notification after a long outage.

Before a scheduled run starts, acquire a lease and create a durable run record. Check whether the same scheduled occurrence already completed. Apply a time budget and tool budget, store a draft, then notify only if there is meaningful new information or a failure requiring attention. These are proposed application controls; verify how they interact with Hermes's scheduler.

Required operational checks

Monitor scheduler heartbeat, memory health, disk space, queue age, inference errors, last successful backup and the availability of required connectors. Give the user pause and cancel controls. A cancel request should stop further tool actions and accurately identify any action that already happened.

Run a 24-hour soak before submission if time permits. Record real timestamps and restart events. A short accelerated schedule is useful for development but must not be presented as proof that a system ran for a day.

15 State durability and recovery

NemoClaw preserves the native home/workspace during routine lifecycle operations. Its rebuild path transfers that state through a bounded archive workflow and sanitizes local authority. Destroy removes managed state; the documented backup operation is not a general snapshot-management API. [S33]

Persistent sandbox state is not an independent backup. It does not protect against accidental deletion, host loss, compromised storage or a bug that writes bad data. Maintain separately encrypted, versioned backups and a restore procedure tested on an isolated dataset.

For SQLite, use a database-safe backup mechanism or quiesced snapshot rather than copying an actively changing database file alone. For a journal with derived indexes, preserve the journal and index checkpoints and demonstrate rebuilding the projections. Avoid live copying of secrets or machine-bound authority between sandboxes.

Define recovery targets as project goals: for example, no loss of acknowledged memory writes under process restart, and restoration of the synthetic demo within 15 minutes from a verified backup. These are proposed acceptance targets, not measured results or framework guarantees.

Test a restore with a corrected fact and a deleted fact. The restored application must serve the corrected value and continue honoring deletion. An apparently successful backup that has never been restored is insufficient evidence.

16 Nebius deployment choices

Two products with different billing models

Token Factory public inference serves provider-managed models through an API. Nebius AI Cloud Serverless runs your container on managed compute. A Token Factory inference call does not create a persistent agent host or a scheduler for your application.

Service	Best role here	Advantage	Constraint
Token Factory public inference	Lightning and Ultra requests	Low setup effort and token billing	Variable capacity and no region guarantee
Token Factory dedicated endpoint	Later regulated or sustained inference	Chosen region and isolated capacity options	Model-specific availability and commercial commitments require checking
AI Cloud Endpoint	Optional application or custom model container	Public request URL and controllable image	Container disk disappears on stop or delete
AI Cloud Job	Finite evaluation or indexing task	Runs to completion with a time budget	No request URL; container data disappears after completion
AI Cloud Devlab	Interactive prototyping	Workspace survives stop and restart	Development lifecycle; stopped disk can still incur charges
Persistent Linux host	Gateway, memory and scheduler	Straightforward persistent service ownership	You operate backups, patching, access and recovery

The storage lifecycle and use cases above are documented in the current Serverless overview. It uses the name Devlab, while hackathon material names DevPods; confirm equivalence if relying on that route for eligibility. The recommended Token Factory path avoids that ambiguity. [S34]

Deployment prerequisites

Serverless creation needs appropriate project IAM permissions, an image, a chosen platform/preset, networking and sufficient quota. The jobs guide supports CPU-only workloads, secret environment variables through SecretStash, bounded timeouts, mounted storage and injected configuration files. [S35]

Do not assume a platform can run NemoClaw's nested container and kernel isolation requirements just because it can run a container. Validate the entire topology. A normal Hermes application container and a managed NemoClaw host are different deployment artifacts.

Use a private registry or pinned public image digest, a dedicated service identity and minimal inbound access. Keep the memory database on suitable persistent storage. Use object storage for exports and backups rather than assuming object storage behaves like a transactional POSIX database volume.

17 Costs and budget controls

Inference scenarios

The following are arithmetic scenarios using the published prices in Section 10, not usage forecasts or measured bills. They exclude caches, retries, background reviews, embeddings, hosting, tax and currency conversion.

Scenario	Input tokens	Output tokens	Lightning cost	Ultra cost
One bounded task	20,000	2,000	$0.00168	$0.026
One hundred such tasks	2,000,000	200,000	$0.168	$2.60
Large evaluation workload	10,000,000	1,000,000	$0.84	$13.00

At these token assumptions, 90 routine tasks on Lightning and 10 difficult tasks on Ultra cost about $0.4112. Escalating after an unsuccessful Lightning attempt adds the cost of that attempt. A multi-turn agent may reread growing history many times, so count all provider requests rather than only the user's initial prompt.

Costs that can dominate

Cloud Serverless uses Compute prices and quotas. Running resources and storage are billed; separately mounted buckets and filesystems remain separately billable. Stopped Devlabs retain disk charges. [S36] A self-hosted GPU running continuously can cost much more than a small number of API requests.

For planning, a hypothetical $0.10/hour host costs $72 for 30 days, while a hypothetical $2/hour GPU costs $1,440. These are examples, not Nebius quotes. Budget continued judging access as well as build-week usage. Obtain a region- and preset-specific quote before selecting paid compute.

Token Factory onboarding currently requires billing setup and a bank card; failed charging can suspend access. [S37] Promotional credit is not a guaranteed hard spending cap. Put limits in your application as well as any account controls available to you.

Proposed controls

Set separate budgets for interactive requests, schedules, background learning and evaluation. Start with a maximum of ten tool turns per routine task, a finite elapsed timeout and a visible cancellation path; tune these values from measurements. Limit Ultra escalation and preserve a queue when a daily background budget is exhausted.

Record model ID, input/output/reasoning usage where reported, request duration and retry count without logging full sensitive prompts. Reconcile estimates against the provider's billing view. Dedicated observability features should not be assumed available on public endpoints; the current observability

guide limits its metrics to dedicated inference. [S38]

18 Test plan and release criteria

Status: this is a proposed validation plan. No application or live-model test was executed in this research task. Use synthetic fixtures, isolated storage and an explicit API-spend budget when implementing it. Keep deterministic tests separate from variable model evaluations.

Core acceptance matrix

Test	How to run it	Evidence and proposed pass condition
Fresh installation	Follow README on a clean supported environment	A second person can reach the demo without undocumented fixes
Provider discovery	List account models and check exact IDs	Both selected IDs exist and respond
Structured tool loop	Ask each model to call a fixed lookup tool	Correct function and arguments; valid result replay
Streaming	Split response and tool arguments across events	Parser reconstructs content and calls without duplication
Empty or reasoning-only reply	Use bounded output and simulated edge responses	Clear incomplete state; no false success
Persistent memory	Save known fact, restart and start a fresh session	Exact approved fact retrieved with origin
Correction	Supersede a fact, query in new and existing contexts	New answer uses current record or explains stale session
Temporal question	Store two dated values and query past/current state	Correct value for requested time
Isolation	Query another user's record through every tool path	Every attempt denied before data retrieval
Deletion	Remove source and memory, rebuild index and restore backup	No active retrieval or resurrection; backup policy disclosed
Duplicate delivery	Replay the same event and operation ID	One durable write and one task occurrence
Index outage	Stop one derived index in the isolated test deployment	Typed degraded result; journal remains correct

Test	How to run it	Evidence and proposed pass condition
Prompt injection	Insert malicious commands into a test paper	No privilege change or unauthorized tool execution
Egress restriction	Request a forbidden destination through multiple clients	Real enforced denial in the chosen sandbox
L7 write restriction	Attempt a blocked HTTP method on an allowed host	Request blocked, not merely logged
Secret canary	Use a fake token in synthetic tool errors and fixtures	Token absent from prompts, public logs and exports
Approval integrity	Change recipient or body after approval	Old approval refused
Provider outage	Simulate 429, timeout and transient 5xx	Bounded retries, no duplicate action, pending work survives
Scheduler restart	Restart gateway around a due occurrence	Defined missed-run behavior and no duplicate notification
Host reboot	Reboot the isolated target host	Documented automatic or manual recovery actually works
Cancellation	Cancel during inference and between tool steps	No new action after cancellation acknowledgement
Backup restoration	Restore into separate storage	Verified facts, corrections and tombstones preserved
Budget exhaustion	Set a tiny background allowance	Background work pauses predictably and reports status
Demo access	Open judging URL in a separate session	Free access and complete testing instructions

Model evaluation design

Create at least 30 fixed synthetic cases: ten retrieval/correction tasks, ten tool-use tasks and ten research synthesis tasks. Give each expected facts, required citations, forbidden actions and a deterministic completion check. Run both models on the same cases with the same evidence bundle and tool permissions. Repeat difficult or variable cases three times if budget allows.

Measure success, source correctness, stale-fact rate, unauthorized action count, schema failures, tool turns, p50/p95 latency and cost per successful task. Report sample size and failure examples. A

model-written assessment can supplement the checks, but should not decide whether its own external action was authorized.

Proposed release targets: zero cross-user disclosures, zero unauthorized external actions, zero false confirmations of durable writes, and all required deterministic acceptance tests passing. Set a task-quality threshold before inspecting final results; 90 percent success on a small benchmark is a possible starting goal, not a universal reliability standard.

Evidence to publish

Publish commands, dependency versions, fixture descriptions, test timestamps and aggregate results. Include redacted provider/model identifiers and actual traces that prove qualifying runtime use. Keep synthetic failures visible. Separate “executed successfully,” “failed,” “not tested” and “unsupported” in the final test report.

19 Main risks and fallback decisions

Risk	Why it matters	Mitigation and fallback
NemoClaw and OpenShell version mismatch	Configuration and inference commands changed	Let the selected NemoClaw installer own its compatible dependency; do not independently upgrade
Managed MCP transport mismatch	Existing memory bridge may expose only stdio or plain HTTP	Add a scoped HTTPS service after a trial, or use direct Hermes tools for the first build
Inference authentication failure	NVIDIA and Nebius keys are different	Verify the selected provider before any agent integration
Country or card onboarding delay	A good application cannot call an unavailable account	Resolve immediately; no alternate-region workaround
Cloud data loss	Disposable container disk holds memory	External persistent store and tested restore
Overstated privacy	Hosted inference receives personal context	Explicit disclosure policy and synthetic public demo
Conflicting memory systems	Correction in one store leaves an older copy active	One authority and revision-aware projections
Background token growth	Reviews and repeated context increase costs	Bound sessions, routes and background budgets
Tool compatibility failure	Compatible API responds but agent cannot use tools	End-to-end structured tool-loop test for both models

Risk	Why it matters	Mitigation and fallback
Reboot interruption	Always-on claim exceeds deployment behavior	Test reboot and disclose manual recovery if necessary
Overbuilding	Four frameworks consume the remaining week	One runtime, one workflow, narrow connectors and a fixed fallback date
Public demo abuse	Shared credentials consume budget or expose records	Separate synthetic tenant, protected admin surface and bounded execution

Specific version finding

The current NemoClaw release note reviewed is v0.0.137 from 20 October. Its managed MCP guide requires OpenShell 0.0.116. The independent OpenShell documentation shows v0.1.5 and says workspace-global managed inference routes were removed in favor of provider profiles and per-sandbox attachments. [S39] [S30] [S40]

These are separate supported paths. Do not combine command examples from them or assume independently installing the newest OpenShell upgrades a NemoClaw deployment safely. Record the exact NemoClaw revision, its resolved OpenShell version, agent image digest, Hermes revision, policy hash, model IDs and tool schema revision in a build manifest.

20 Build plan through submission

The dates below are a recommended plan starting 23 October. Reorder around actual progress. The latest safe point to simplify is before the final integration days.

Day	Outcome	Completion evidence
23 October	Account access, scope and architecture trial	Both model calls, one actual tool loop, chosen runtime and written fallback decision
24 October	Durable memory and provenance	Save, fresh-session recall, correction, scope filtering and deletion fixtures
25 October	Useful research workflow	Chosen documents to cited brief, saved decision and draft artifact
26 October	One scheduled workflow	Real next-run status, pause/cancel, missed-run handling and persistent execution record
27 October	Security and failure testing	Negative egress, approval tampering, injected document, outage and restore results

Day	Outcome	Completion evidence
28 October	Product polish and soak	Clear onboarding, memory inspector, disclosure view and actual availability log
29 October	Submission freeze and rehearsal	Public clean repository, clean setup rehearsal, video, screenshots and feedback
30 October	Final verification and submission	Working links, access instructions and submitted entry before the internal cutoff

Set an internal deadline several hours before 22:30 India time. Freeze features once the core workflow is credible. Spend the final day checking links, account balance, reproducible setup, public artifact privacy, and the demonstration path.

Three minute demonstration plan

Use about 20 seconds to show the user's problem and selected sources; 40 seconds to create a cited brief and durable decision; 30 seconds to start a fresh session and demonstrate recall; 25 seconds to correct a fact; 25 seconds to show a scheduled run and its actual output; 20 seconds to show a blocked malicious instruction and exact-payload approval; and the remaining time to show the model/provider trace, architecture and measured limitations.

Use real footage of the application. Label any shortened schedule, synthetic dataset or replay. Show one strong complete workflow rather than a long list of disconnected screens. Make the source repository and setup instructions easy to locate.

Repository and submission package

Prepare application code, a clear license, third-party notices, a setup guide, an example configuration without secrets, synthetic fixtures, a build manifest and a concise architecture document. Include an explanation of meaningful additions to any pre-existing memory code. Provide a test report that distinguishes deterministic checks from live-model evaluations and lists known limits.

Describe the personal workflow, the user's controls, where Nebius inference runs, why the two model tiers are useful, and what the demo actually proves. Keep judge access available through the published judging period and maintain a recovery contact/process. Do not publish real user credentials, private papers without rights, personal memory exports or debug logs containing prompts.

21 Decisions still requiring validation

Account and commercial checks

Confirm Token Factory onboarding, country and payment acceptance, both model IDs, actual per-model limits, credit balance and expiry. Confirm the applicable zero-retention mode and terms, public processing geography, dedicated model availability if needed, and whether any provider telemetry stores prompt content. Price the exact cloud resources before deployment.

Technical checks

Confirm the existing memory_system's storage authority, supported API, authentication, ownership boundaries, migration behavior and restore procedure. Determine whether it can expose the proposed scoped tool contract without broad database credentials. Run its actual current test suite in the main development task before treating it as a foundation.

Validate the chosen host and pinned NemoClaw/Hermes/OpenShell combination. Verify a Token Factory tool call from inside the sandbox, the memory endpoint's network path, scheduler behavior under restart, and all configured auxiliary model routes. Confirm whether the desired app UI can surface approval and cancellation faithfully.

Questions for organizers if relevant

If using an AI Cloud Devlab instead of Token Factory for eligibility, request clarification about the DevPod/Devlab naming. If substantial existing work predates the event, describe its new contributions precisely and ask if the scope is unclear. If prize or country eligibility is uncertain, ask through the event's official channels before entry. No messages were sent during this research.

Foundation decision record

Record the decision, date, evidence, alternatives rejected, unresolved assumptions and trigger for revisiting it. Recommended triggers include repeated tool failures, a changed provider data policy, loss of a required model, unacceptable recall errors, an inability to delete data completely, or a host that cannot meet the claimed availability.

22 Build prerequisites and starter runbook

What to prepare

Prepare a Devpost account, an approved Nebius account with working billing and inference credit, a public code repository, an operating budget, and a separate synthetic demonstration dataset. For the proposed managed stack, reserve a supported Linux host with at least 4 vCPU, 8 GB RAM and 20 GB free disk; the NemoClaw guide recommends 16 GB RAM and 40 GB free disk. It lists Node 22.19 or later and npm 10 or later. These are agent-stack requirements, not model-hosting capacity. [S09]

A GPU is unnecessary on the agent host when inference is remote. Add capacity for the actual memory databases, document processing and backups. Keep the model host, agent host and memory host

conceptually separate even if two roles initially share one machine. For Windows development, use Hermes's supported native distribution or the documented WSL2 path for NemoClaw, with its platform caveats. [S07] [S06]

The application also needs an identity and session boundary, selected-source ingestion, a memory API, a draft workspace, an approval store, scheduler status, logging, backup storage and a user interface. Every connector added to this list needs its own account permission, disclosure decision and failure test. Create an inventory showing where each credential lives and which process can use it.

Start with provider discovery

The following Python example uses the standard library and the documented Token Factory discovery route. It is a proposed read-only diagnostic, not a test run performed for this report. It makes one authenticated network request when executed. Supply the key through the approved environment; do not paste it into the file. [S18]

```
import json
import os
import urllib.request

url = "https://api.tokenfactory.nebius.com/v1/models"
request = urllib.request.Request(
    url,
    headers={"Authorization": "Bearer " + os.environ["NEBIUS_API_KEY"]},
)
with urllib.request.urlopen(request, timeout=30) as response:
    models = json.load(response)

wanted = {
    "nvidia/Nemotron-3_5-Lightning",
    "nvidia/Nemotron-3-Ultra-550b-a55b",
}
available = {item["id"] for item in models["data"]}
print({model: model in available for model in sorted(wanted)})
```

A missing ID is a stop condition for that route, not a reason to guess a different spelling. Save the discovery timestamp and a redacted metadata snapshot. Then perform separate paid completion and tool-call checks using the provider's tested request format.

Configuration map

Setting	Intended value	Verification
Inference base URL	https://api.tokenfactory.nebius.com/v1/	Actual request reaches this provider
Routine model	Exact Lightning ID above	Model reported in redacted request trace
Difficult model	Exact Ultra ID above	Explicit task-level route with a budget

Setting	Intended value	Verification
API protocol	Chat Completions for the initial integration	Completion and tool-result replay both pass
Memory writes	Approved proposals to the chosen authority	Durable receipt and fresh-session recall
Tool scope	Selected sources and draft workspace	Negative tests outside the allowed scope
Unattended actions	Deny dangerous commands; stage external writes	No unattended approval bypass
Auxiliary inference	Explicit allowed providers and models	Compression, review and title routes inspected

Use the installed version's Hermes model configuration UI or documented custom-provider schema. Avoid treating a generic YAML example as a universal configuration file. Hermes documents independent auxiliary routes and credential resolution; a base URL override can change where an auxiliary task sends data. [S41]

For a NemoClaw-managed setup, use its own onboarding and lifecycle commands to create the compatible environment. Enter the hosted custom endpoint and exact model explicitly. New default onboarding may select a different model from the one this project intends to demonstrate. [S08] [S39]

Hermes defaults dangerous-command handling to smart mode with an auxiliary model, and documents deny behavior for cron and other unattended contexts. Preserve explicit application approvals for external consequences even when shell command checks pass. Use the supported configuration for the chosen version and test the effective behavior. [S42]

Daily operating runbook

Before a demo, verify account balance, provider health, memory read/write health, scheduler heartbeat, free disk and the last verified backup. Run one harmless actual tool loop and inspect the final result. Check the public demonstration environment from a fresh browser session.

If inference fails, preserve the pending request and diagnose account, model, DNS, timeout and provider status separately. If memory is degraded, refuse durable-save success and avoid confident personalized answers from incomplete evidence. If the host restarts, follow the selected stack's documented recovery sequence and check jobs before resuming notifications.

Before changing versions, create a verified backup, record the old manifest, upgrade an isolated copy and run the core acceptance matrix. During the submission week, avoid unrelated upgrades once the demonstration stack is stable.

23 Source ledger and review coverage

All sources below were accessed on 23 October 2026. “Page body” means the substantive short page was reviewed; it excludes footer links and linked material. “Selected sections” identifies focused review of a longer page. It does not mean every example, submenu, release or linked source was audited.

S01 Hackathon overview — requirements, submission list, track and prizes. Selected sections. Open source

S02 Official rules — project/submission rules, schedule, judging, access, ownership and terms. Substantive rules reviewed across retrieved sections; no personal eligibility determination. Open source

S03 Event FAQ — platform eligibility, credits, countries, teams and city attendance. Page body. Open source

S04 Event resources — credit routes and official learning resources. Page body. Open source

S05 NemoClaw ecosystem — component roles and custom OpenShell comparison. Page body. Open source

S06 NemoClaw platform support — alpha status, agents, Windows/Linux, provider and unsupported deployment rows. Selected sections. Open source

S07 Hermes platform support — supported distributions and operating systems. Page body. Open source

S08 NemoClaw Hermes quickstart — onboarding, routing, state and lifecycle. Selected sections; installation starter prompt treated as source content, not executed. Open source

S09 NemoClaw prerequisites — CPU, RAM, storage, runtime and dependency requirements. Selected sections. Open source

S10 NemoClaw provider selection — hosted/custom/local provider support. Page body. Open source

S11 Hermes memory — file limits, frozen context, session search, approval and review settings. Selected sections. Open source

S12 Hermes memory providers — additive behavior, transcript synchronization and secret-scrub limits. Selected sections. Open source

S13 Hermes skills — storage, progressive loading and security scanning. Selected sections. Open source

S14 Nebius Nemotron models — prices, serving precision, context, catalog links and model choice. Selected sections. Open source

S15 NVIDIA Lightning model card — summary, hardware matrix and license. Selected sections. Open source

S16 NVIDIA Ultra BF16 model card — official search-visible summary including parameters, license and GPU requirements; full quantized card fetch exceeded tool limits. Partial review. Open source

S17 Hermes model configuration — main/auxiliary slots and cache effects of switching. Selected sections. Open source

S18 Token Factory model discovery — API and verbose metadata. Page body. Open source

S19 Nebius agent cost cookbook — methodology, published example and Ultra harness caveat. Page body. Open source

S20 Token Factory quickstart — endpoint and credential convention. Selected sections. Open source

S21 NemoClaw custom endpoint — adapter, validation, route and private networking limitations. Page body. Open source

S22 Token Factory tool calling — execution ownership, schema and request flow. Page body. Open source

S23 Token Factory structured output — formats, model-specific support and schema guidance. Selected sections. Open source

S24 Token Factory rate limits — dynamic limits, account defaults and burst behavior. Selected sections. Open source

S25 Nebius enterprise inference — zero-retention statements and plan distinctions. Selected sections; marketing statements are not an account-specific contract. Open source

S26 Token Factory public endpoints — global routing and no guaranteed region. Page body. Open source

S27 OpenShell architecture — supervisor, sandbox, policy and credential boundary. Selected sections. Open source

S28 OpenShell security — enforcement, audit/enforce, TLS and access boundaries. Selected sections. Open source

S29 Hermes MCP — transport, catalog setup and tool exposure. Selected sections. Open source

S30 NemoClaw managed MCP — accepted architecture, pinned dependency, credentials, transport and limitations. Page body. Open source

S31 Hermes cron — gateway scheduler, health, durable storage and restart-safe worker conditions. Selected sections. Open source

S32 NemoClaw headless deployment — resources, loopback access, pinned installation and manual reboot recovery. Selected sections. Open source

S33 NemoClaw persistence — routine persistence, rebuild archive, backup and destroy. Page body. Open source

S34 Nebius Serverless overview — deployment types, storage lifecycle and billing behavior. Page body. Open source

S35 Nebius Serverless jobs — prerequisites, CPU/GPU selection, secrets, storage and timeouts. Selected sections. Open source

S36 Nebius Serverless pricing and quotas — Compute billing, stopped resources and separate mounted storage. Page body. Open source

S37 Token Factory billing — mandatory card setup and charge/suspension behavior. Selected sections. Open source

S38 Token Factory observability — metric access limitation, latency, errors and data locality. Page body. Open source

S39 NemoClaw 20 October release note — v0.0.137 and defaults. Page body. Open source

S40 OpenShell inference — native providers, endpoint binding, verification and removed managed routes. Page body. Open source

S41 Hermes configuration — custom endpoint and auxiliary credential-routing sections. Selected sections only; full reference not audited. Open source

S42 Hermes security — approval modes, unattended behavior and MCP credential filtering. Selected sections. Open source

Supplied portals and discovery coverage

The supplied NemoClaw OpenClaw home, Hermes documentation home, OpenShell home, Token Factory inference overview and Nebius cloud home were reviewed as orientation pages. Their relevant child pages are identified above. The supplied Hermes quickstart is S08. The hackathon link is S01. Documentation indexes from all five documentation families were retrieved for discovery; this does not establish review of all indexed pages.

Coverage gaps and unresolved access

The authenticated model catalog, account limits, actual credit approval and deployment availability were not inspected. A standalone OpenShell support-matrix fetch failed, so this report relies on the explicit NemoClaw matrix for its managed path and does not infer universal OpenShell host support. The Ultra NVFP4 card exceeded retrieval limits; its complete runtime recipes were not reviewed. Some cached-token and versioned Markdown URLs failed. No linked video or slide deck was reviewed in full. No exact cloud price quote, live inference benchmark, current repository audit, or personal eligibility approval was obtained.

How to update this research

Before implementation, refresh the event rules and FAQ, model discovery metadata, provider prices, selected version's security notes, host support and data policies. Update this report's date and decision record when a material fact changes. Preserve the old snapshot so changes in the foundation remain traceable.